Static and Molecular-Dynamics-Derived Residue Graphs for Protein–Protein Binding Affinity Prediction

Student Name

The thesis is submitted to University College Dublin
in part fulfilment of the requirements for the degree of
BSc Applied and Computational Mathematics

School of Mathematics and Statistics
and
Supervisor's School (if different)
University College Dublin

Supervisor: Supervisor Name
Second Supervisor Name (if applicable)

March 17, 2026

Abstract

Protein–protein binding affinity prediction is an attractive target for machine learning because affinity is experimentally measurable, biologically important, and directly relevant to therapeutic design. At the same time, it is technically difficult: affinity depends on a three-dimensional interface, long-range structural context, solvent exposure, and conformational dynamics, while available structure–affinity datasets remain comparatively small. This thesis investigates whether short explicit-solvent molecular dynamics simulations can improve residue-graph affinity prediction when compared with static structural features alone.

The active pipeline begins with a curated wild-type subset of the PPB-Affinity dataset and processes each complex through explicit-solvent molecular dynamics, residue-level graph construction with DeepRank2, and downstream learning with either a masked graph variational autoencoder or a direct supervised graph regressor. Two graph views are considered: a full-complex residue graph and a fixed-size interface patch. Two feature modes are also compared: a static representation (S) and a static-plus-dynamic representation (SD), where the dynamic channels summarize hydration, Cα flexibility, backbone torsions, contact persistence, distance statistics, and approximate inter-chain nonbonded force proxies. Final evaluation is performed on a matched 1437-complex subset using a repeated outer resampling design with 52 model configurations, 5 folds, 2 repeats, and 520 total fits.

The confirmatory results are clear. The full graph view outperforms the interface view in all 24 matched variational-model comparisons, with a mean interface penalty of 0.1005 test-RMSE units. The dynamic augmentation does not yield a robust held-out gain over the static representation; in 11 of 12 matched S versus SD comparisons under the shared-static reconstruction policy, the static model performs better. The strongest overall model is the direct supervised baseline on the full static graph, with mean test RMSE 2.3379, mean test Pearson correlation 0.5064, and mean test R^2 0.2146. Semi-supervised dynamic models fit the training data more strongly but also show the largest train–test gaps, indicating increased flexibility without a commensurate gain in generalization.

The main conclusion is therefore not that protein dynamics are unimportant, but that the present dynamic encoding strategy is not strong enough to improve affinity prediction over a well-constructed static residue graph. The thesis contributes a fully audited end-to-end pipeline, a controlled comparison of graph view and feature mode, and a robust negative result that is scientifically useful precisely because it survives repeated held-out evaluation.

Acknowledgments

I am grateful to my supervisor for guidance throughout the project and for repeated feedback on both the technical work and the written thesis. I also thank the staff of the School of Mathematics and Statistics and the collaborating research environment that made the computational work possible. Finally, I thank my family and friends for their support during the project.

Contents

1 Introduction **1**

1.1 Problem setting and motivation 1

1.2 Research questions and scope 2

1.3 Contribution of the thesis 3

1.4 Thesis structure 3

2 Literature Review **4**

2.1 Protein–protein affinity prediction as a distinct modelling problem 4

2.2 Physics-based routes to binding free energies 5

2.3 Empirical and structure-based models for protein–protein affinity 6

2.4 Why the protein–ligand literature is ahead 6

2.5 Variational autoencoders in molecular and structural learning 7

2.6 Where this thesis fits 8

3 Background and Theoretical Context **9**

3.1 Protein–protein binding affinity as a learning target 9

3.2 Why molecular dynamics may add information 10

3.3 Residue graphs as representations of protein complexes 10

3.4 Graph neural networks and message passing 11

3.5 Variational autoencoders and masked reconstruction 12

3.6 Why repeated outer resampling matters 12

4 Methods **14**

4.1 Dataset curation and affinity labels 14

4.2 Molecular dynamics system preparation 15

4.3 Molecular dynamics protocol 15

4.4 Residue-graph construction 16

4.5 Molecular dynamics-derived node and edge features 16

4.6 Approximate inter-chain force features 17

4.7 Graph views and matched analysis subset 17

4.8 Feature modes and preprocessing 18

4.9 Masked graph variational autoencoder . . . 18
4.10 Direct supervised baseline . . . 19
4.11 Latent-space regression and final VAE evaluation . . . 20
4.12 Repeated outer resampling and model matrix . . . 20
4.13 Evaluation metrics . . . 21

5 Results **22**
5.1 Headline model performance . . . 22
5.2 Effect of graph view: full versus interface . . . 23
5.3 Effect of dynamic features: S versus SD . . . 24
5.4 Effect of supervision . . . 25
5.5 Effect of reconstruction target policy . . . 25
5.6 Overfitting and train–validation–test behaviour . . . 26
5.7 Summary of empirical conclusions . . . 27

6 Discussion and Conclusions **28**
6.1 Principal findings . . . 28
6.2 Why the full graph outperformed the interface graph . . . 28
6.3 Why the present dynamic features did not help . . . 29
6.4 Why the direct supervised baseline was strongest . . . 30
6.5 What the train–validation–test behaviour shows . . . 30
6.6 Limitations and threats to validity . . . 31
6.7 Future work . . . 31
6.8 Conclusion . . . 32

A Supplementary Technical Background **33**
A.1 Molecular dynamics in plain terms . . . 33
A.2 Residue graphs and graph neural networks in plain terms . . . 34
A.3 Variational autoencoders in plain terms . . . 34
A.4 Why use ridge regression on the latent space . . . 35
A.5 Why repeated outer resampling matters in practical terms . . . 35

B Archived Approaches and Project Evolution **37**
B.1 Correction learning around fast affinity baselines . . . 37
B.1.1 PRODIGY . . . 38
B.1.2 MM/GBSA . . . 38
B.2 The Cα contact-map branch . . . 39
B.3 The PPB-inspired augmentation and spatiotemporal branch . . . 40
B.4 Exploratory dataset curation utilities . . . 41
B.5 What the archived branches contributed . . . 42

C Repository and Implementation Availability **43**

List of Figures

List of Tables

5.1 Selected headline models from the final 52-configuration confirmatory matrix. Metrics are means over 10 outer held-out fits on the matched 1437-complex set. For GraphVAE models, the reported predictive metrics come from the downstream ridge probe on latent means rather than solely from the neural affinity head. 23

5.2 Family-level train/validation/test behaviour. Each row is the arithmetic mean over all configurations in that family, using the configuration-level means from the 10 outer held-out fits. Here $g_r = \bar{r}_{\text{train}} - \bar{r}_{\text{test}}$. 27

Chapter 1

Introduction

1.1 Problem setting and motivation

Protein–protein interactions are central to cellular signalling, immune recognition, enzyme regulation, and macromolecular assembly. In many scientific and therapeutic settings, the question is not simply whether two proteins bind, but how strongly they bind. The relevant thermodynamic quantity is the binding free energy, or equivalently the dissociation constant K_D, which determines how readily a bound complex forms and dissociates under equilibrium conditions. Predicting this quantity from structure is an attractive machine-learning problem because it is concrete, experimentally measurable, and relevant to biomolecular design. However, it is also a difficult one. The determinants of binding affinity are distributed across geometry, electrostatics, solvent exposure, local flexibility, and long-range structural context, and experimentally labelled protein–protein datasets remain limited compared with the scale of protein sequence and structure space.

The recent PPB-Affinity dataset was created specifically to support machine-learning research on protein–protein binding affinity prediction [1]. It provides a scale and level of curation that make systematic benchmarking possible. At the same time, the existence of a large dataset does not remove the underlying modelling challenges. A static crystal or cryo-EM structure captures only one experimentally observed conformation of a complex, whereas binding is a thermodynamic process taking place over an ensemble of solvated molecular states. This motivates a central question for the present thesis: can short molecular-dynamics simulations provide useful additional information beyond a carefully engineered static structural graph?

Graph-based learning is a natural way to pose this question. A protein complex can be represented as a residue-level graph in which residues form nodes and spatially or physicochemically related pairs form edges. Static structural features can then be assigned to nodes and edges, and graph neural networks can be used to learn graph-level embeddings for affinity prediction [2–5]. Such methods are attractive because they can

combine local chemical information with higher-order structural context. However, they are still typically built from a single structure or from features extracted from a single frame. Proteins are not rigid objects, and the importance of conformational fluctuations and dynamic ensembles in biomolecular function has long been recognized [6]. Hydration, contact persistence, and backbone flexibility may therefore carry predictive information not present in a static graph alone.

The appeal of dynamics comes with a difficulty. Rich trajectory data are expensive to generate, high-dimensional to represent, and easy to summarize badly. It is therefore not enough to ask whether "dynamics help" in the abstract. One must ask whether a specific dynamic representation, with a specific simulation budget and a specific graph-learning pipeline, improves held-out predictive performance relative to a strong static baseline. That is the problem addressed here.

1.2 Research questions and scope

This thesis focuses on the final active pipeline implemented in the repository rather than on earlier exploratory branches that were later archived. The workflow begins with a curated broad subset of PPB-Affinity, prepares each complex for explicit-solvent molecular dynamics, constructs residue graphs with DeepRank2, augments these graphs with molecular-dynamics-derived descriptors, and trains either a masked graph variational autoencoder or a direct supervised graph regressor. The evaluation is restricted to a matched subset of complexes for which both the full-complex graph and the interface-only graph can be materialized successfully, allowing a controlled comparison of graph view and feature mode.

Within this scope, the thesis addresses four specific questions. First, is a full residue graph more predictive than a local interface graph for the present dataset and feature set? Second, do molecular-dynamics-derived node and edge summaries improve predictive performance relative to static graph features alone? Third, does a representation-learning approach based on a masked graph variational autoencoder outperform a direct supervised graph regressor when both are given the same graph inputs? Fourth, what can be learned from the train/validation/test behaviour of these models about underfitting, overfitting, and the reliability of conclusions drawn from one-off train/test splits?

These questions are deliberately narrower than the general question of whether molecular dynamics can ever help affinity prediction. The thesis does not attempt to compute rigorous free energies, perform long-timescale conformational sampling, or exhaustively benchmark all possible graph constructions. Instead, it studies a well-defined and fully reproducible pipeline and asks what can be concluded about that pipeline from a careful confirmatory evaluation.

1.3 Contribution of the thesis

The first contribution of the thesis is engineering: a complete end-to-end pipeline from curated structure–affinity data to graph-based prediction. This includes automated preprocessing of protein complexes, explicit-solvent molecular dynamics in OpenMM [7], residue-level graph construction using DeepRank2 [2], dynamic feature extraction with MDTraj [8], and two downstream model families. The second contribution is methodological: the thesis compares full and interface graph views, static and static-plus-dynamic feature sets, unsupervised and semi-supervised variational training, and direct supervised baselines under a repeated outer resampling design rather than relying on a single split. The third contribution is scientific: the final results provide a robust negative result for the current dynamic encoding, while also clarifying that the negative result arises from the present representation choice rather than from a principled argument that dynamics are irrelevant.

This last point is important. In a project of this kind, a carefully established negative result is often more informative than an optimistic but fragile positive one. A claim that dynamic features help on one train/test split can be overturned by resampling. A claim that they do not help in a specific, audited pipeline is narrower, but more trustworthy. The thesis therefore aims not only to build a predictive model, but also to demonstrate sound scientific reasoning about what the model can and cannot support.

1.4 Thesis structure

Chapter 2 places the thesis in the context of existing work on protein–protein affinity prediction, molecular simulation, structural learning, and variational representation learning. Chapter 3 then provides the technical background needed for the rest of the thesis, including molecular dynamics, residue graphs, graph neural networks, and variational autoencoders. Chapter 4 describes the implemented pipeline in replication-level detail, including dataset curation, simulation settings, graph construction, model definitions, and the exact confirmatory resampling protocol. Chapter 5 reports the final 52-configuration evaluation matrix and quantifies the effects of graph view, feature mode, supervision, target policy, and model family. Chapter 6 interprets these findings, discusses their limitations, and states the main conclusions of the thesis. The appendices provide additional technical explanation and a map from the written thesis to the concrete code and data artifacts in the repository.

Chapter 2

Literature Review

2.1 Protein–protein affinity prediction as a distinct modelling problem

Predicting protein–protein binding affinity occupies an awkward position between structural bioinformatics, computational chemistry, and machine learning. It is structurally informed, because the arrangement of residues at and around the interface plainly matters. It is also thermodynamic, because the target quantity is the free energy of association rather than a purely geometric score. That dual character makes the problem harder than many standard structure-to-property tasks. A crystal or cryo-EM complex gives one experimentally observed structure, but the measured affinity reflects a balance of enthalpic and entropic effects across an ensemble of bound and unbound states. Solvent, conformational heterogeneity, and local flexibility all matter, yet they are only partially visible in a single deposited structure.

The practical literature reflects this difficulty. In protein–protein learning, a great deal of work has historically focused on interface classification, docking-model quality assessment, hotspot prediction, or mutation-induced changes in binding energy, rather than direct prediction of absolute binding affinity for a complex. The popularity of SKEMPI-type benchmarks illustrates this point: curated mutation datasets made the supervised prediction of $\Delta\Delta G$ more tractable than the direct prediction of absolute ΔG for arbitrary complexes [9]. Absolute affinity prediction has therefore had a thinner benchmark ecosystem, a narrower set of directly comparable models, and less methodological convergence than adjacent tasks.

This context matters for the present thesis. The question is not simply how to squeeze another fraction of a kcal/mol out of a leaderboard. The more basic question is what representation and what modelling compromise are scientifically reasonable when one wants to predict protein–protein affinity from structure while still keeping the pipeline computationally feasible and auditable.

2.2 Physics-based routes to binding free energies

The most physically direct route to binding affinity is to estimate free-energy differences from molecular simulation. In principle, this is attractive because it targets the quantity of interest directly rather than asking a learned model to approximate it indirectly. In practice, however, the available methods span a wide range of cost, approximation, and implementation complexity.

At the more rigorous end are enhanced-sampling and alchemical approaches. Umbrella sampling is a classic example. It introduces biasing potentials along a chosen reaction coordinate so that rare but important configurations are sampled more often, after which the unbiased free-energy profile is reconstructed [10]. Alchemical binding free-energy methods go further by defining nonphysical thermodynamic pathways between states and integrating free-energy changes along those paths. These methods have become increasingly mature, especially in protein–ligand problems, and they can be highly informative when carefully set up [11]. Their drawbacks are equally well known: they are expensive, sensitive to protocol design, require substantial expertise, and depend strongly on sampling quality and force-field fidelity. For large protein–protein complexes, these difficulties are magnified by system size, interface flexibility, solvent exposure, and the challenge of defining robust collective variables or alchemical pathways.

At the cheaper end are endpoint methods such as MM/PBSA and MM/GBSA. These combine molecular mechanics energies with continuum-solvent approximations and are widely used as approximate binding estimators [12]. Their appeal is obvious: they are much cheaper than full free-energy calculations and easy to layer onto existing MD workflows. Their limitations are equally important. They depend on modelling assumptions about solvation, entropy, and state averaging, and their absolute predictive accuracy can vary markedly across systems and protocols [12]. In other words, they are often useful comparison points, but they do not remove the underlying difficulty of the affinity problem.

This physics literature influenced the design of the present thesis in two ways. First, it motivates taking molecular dynamics seriously: if conformational fluctuations and solvent behaviour matter physically, then a purely static model is at least open to question. Second, it also motivates restraint. Full rigorous free-energy calculations are not a realistic centrepiece for a thesis whose main goal is to compare learned structural representations at scale. That is why the present work uses MD as a source of descriptors rather than as a direct free-energy engine.

2.3 Empirical and structure-based models for protein–protein affinity

Because rigorous simulation is expensive and endpoint methods remain approximate, a parallel literature has pursued empirical or machine-learned affinity estimation from structure. One representative example is PRODIGY, which predicts protein–protein binding affinity from interfacial contact counts and related structural descriptors [13]. The attraction of such models is that they are fast, interpretable, and easy to apply across large datasets. The drawback is that they compress a complex physical process into a small number of hand-designed descriptors. That can work surprisingly well in some settings, but it also limits the expressive capacity of the model.

The recent PPB-Affinity dataset is an important development in this space because it was assembled specifically to support AI-based protein–protein affinity prediction [1]. Its existence signals that the field has moved beyond isolated small benchmarks and toward more systematic evaluation. At the same time, the dataset paper makes clear why the task remains challenging: the available structures are heterogeneous, the complexes span multiple biological regimes, and the number of experimentally labelled cases remains small compared with typical deep-learning scales [1]. The problem is therefore large enough to motivate machine learning, but still data-limited enough that representation choice matters acutely.

Recent geometric-learning frameworks such as DeepRank2 show that protein interfaces can be represented effectively with residue-level or atom-level graphs equipped with biophysical node and edge features [2]. That literature is highly relevant even when it is not aimed exclusively at absolute affinity prediction, because it demonstrates a general shift in structural bioinformatics away from rigid handcrafted scoring functions and toward learned geometric representations of macromolecular structure. In this thesis, that shift is taken seriously, but with a narrower question in mind: whether a residue-graph representation augmented with short-trajectory descriptors can outperform an otherwise comparable static representation.

2.4 Why the protein–ligand literature is ahead

A useful way to understand the state of protein–protein affinity prediction is to compare it with the protein–ligand case. Protein–ligand affinity prediction is still difficult, but it benefits from a much longer benchmarking tradition, more standardized datasets, and a clearer methodological pipeline from structure to score. PDBbind is the obvious example of this infrastructure: it provides a large collection of protein–ligand complexes with associated binding measurements and has become a standard reference point for supervised affinity modelling [14]. Once such infrastructure exists, it becomes easier to compare

classical scoring functions, 3D convolutional models, graph networks, and MD-enhanced methods on roughly common ground.

This difference in ecosystem partly explains why the protein–ligand literature has moved faster on learned structural models. Pafnucy is a well-known example of a deep learning model that uses a regularized spatial representation to predict protein–ligand binding affinity [15]. More recently, spatio-temporal learning directly on MD-derived protein–ligand data has also been explored, including models that use trajectory information rather than a single static input [16]. The protein–ligand case therefore provides a proof of principle that geometric and even trajectory-aware learning can be worthwhile when the data and the representation support it.

Protein–protein complexes are harder in several practical respects. The interfaces are larger, the partners are flexible on both sides, the systems are more heterogeneous, and the data are less standardized. Even the choice of representation is less obvious: a ligand has a natural identity as a small graph or 3D object inside a binding pocket, whereas a protein–protein complex raises immediate questions about whether to represent the whole assembly, only the interface, or some fixed local patch. This is one reason the present thesis compares full-complex and interface-only views directly rather than treating the interface representation as self-evidently correct.

2.5 Variational autoencoders in molecular and structural learning

Variational autoencoders were introduced as a way to learn continuous latent-variable models by combining stochastic encoding, reconstruction, and variational regularization [17]. In molecular machine learning, they became attractive not only for generation but also for representation learning: a VAE can force a model to compress high-dimensional structured inputs into a smoother latent space while still preserving information needed for reconstruction or downstream prediction. This idea has been influential in molecular design and molecular graph modelling, for example in continuous latent-space approaches to chemical generation [18] and in graph-based variational models such as GraphVAE and variational graph autoencoders [19, 20].

What matters for the present thesis is not molecular generation itself, but the representational role of the VAE. Protein–protein affinity datasets are not large enough to make unrestricted end-to-end supervised learning an obviously optimal choice in every regime. A variational graph model offers a different compromise. It can use reconstruction as an auxiliary task, exploit unlabeled or weakly labeled structure in the data representation, regularize the latent space, and still support downstream supervised evaluation. In other words, the VAE is appealing precisely when one suspects that the input representation

contains more structure than the label budget alone can reliably exploit.

At the same time, VAEs are not the dominant standard in direct protein–protein affinity prediction. The literature on affinity prediction is still led more by supervised regression models than by latent-variable approaches. That makes the present thesis somewhat exploratory in how it uses the VAE. The model is not introduced because the literature has already converged on VAEs as the canonical tool for this task. It is introduced because variational representation learning offers a principled way to test whether graph reconstruction and affinity prediction can share a useful latent structure when the labels are limited and the feature space is rich.

2.6 Where this thesis fits

Taken together, the literature suggests a gap rather than a settled recipe. Purely physics-based methods are principled but often too costly or too approximate to serve as the sole practical solution at the scale of modern structural datasets. Purely empirical structure-only scorers are cheap and sometimes effective, but they may compress away information that matters. Protein–ligand work shows that geometric and even trajectory-aware learning can succeed when the data and benchmark culture are strong enough, but the protein–protein case remains less mature. Variational latent models are established tools for molecular representation learning, but they are not yet a standard answer to protein–protein affinity prediction.

The thesis sits in that gap. It does not try to beat rigorous free-energy methods at their own objective, and it does not assume that a single static structural graph must already contain everything worth knowing. Instead, it asks a more focused question: if one starts from a strong residue-graph representation of a protein complex, can short explicit-solvent molecular dynamics be distilled into graph-compatible descriptors that improve held-out affinity prediction? The choice of a graph representation, the inclusion of dynamic summaries, the comparison between full and interface graph views, and the use of a masked GraphVAE alongside a direct supervised baseline all follow naturally from that question. The literature does not dictate that this pipeline should work. It does, however, make the question worth asking.

Chapter 3

Background and Theoretical Context

3.1 Protein–protein binding affinity as a learning target

The quantitative target in this thesis is the standard Gibbs free energy of protein–protein binding, ΔG. If a complex is written schematically as

$$\mathrm{P}_1 + \mathrm{P}_2 \rightleftharpoons \mathrm{P}_1\mathrm{P}_2, \tag{3.1}$$

then the equilibrium dissociation constant is

$$K_D = \frac{[\mathrm{P}_1][\mathrm{P}_2]}{[\mathrm{P}_1\mathrm{P}_2]}, \tag{3.2}$$

and the corresponding free energy is

$$\Delta G = RT \ln K_D. \tag{3.3}$$

This relation matters for two reasons. First, it makes it possible to combine experimental measurements reported in different but thermodynamically compatible formats by converting them to a single continuous regression target. Second, it highlights that affinity is not merely a geometric property of one structure. It is a thermodynamic quantity summarizing the balance of enthalpic and entropic effects across a whole ensemble of conformations and solvent configurations.

For machine learning, this means that affinity prediction from structure is intrinsically a many-to-one problem. A deposited structure provides a highly informative but incomplete snapshot. The model must learn to map that snapshot, or features derived from it, onto a quantity that depends on a broader physical ensemble. This tension is one of the reasons affinity prediction is challenging even when reasonably large curated datasets are available [1].

3.2 Why molecular dynamics may add information

Molecular dynamics (MD) simulation provides a controlled way to generate short trajectories around an experimentally observed structure. In classical MD, atomic positions evolve under Newton's equations of motion,

$$m_i \frac{d^2 \mathbf{r}_i}{dt^2} = \mathbf{F}_i = -\nabla_{\mathbf{r}_i} U(\mathbf{r}_1, \ldots, \mathbf{r}_N), \tag{3.4}$$

where U is the force-field potential energy and $\mathbf{F}_i$ is the force on atom i. In explicit-solvent simulation, water molecules and ions are included directly, allowing one to observe fluctuations in local hydration, residue mobility, and intermolecular contact geometry. These are all plausible carriers of affinity-relevant information.

The motivation for including MD in this thesis is therefore not to replace structure-based learning with full free-energy simulation. The goal is more modest and more practical: to test whether short explicit-solvent trajectories can provide descriptors that are informative enough to improve affinity prediction. This is a descriptor-based use of MD, not a direct free-energy calculation. The distinction matters. Free-energy methods aim to approximate a thermodynamic quantity itself, often at substantial computational cost. Descriptor-based approaches instead ask whether trajectory-derived statistics such as residue flexibility, hydration, or contact persistence can improve a learned predictor. These two uses of MD should not be conflated.

This motivation is physically plausible because proteins are dynamic molecules rather than rigid objects. Functionally relevant conformational fluctuations, transient contacts, and solvent rearrangements can matter even when the average structure remains similar [6]. At the same time, the usefulness of MD-derived descriptors is not guaranteed. Their value depends on simulation length, force-field accuracy, feature design, and how they are integrated with the learning model. One of the central empirical questions of the thesis is whether a coarse but computationally tractable dynamic summary is sufficient to improve performance over a strong static residue graph.

3.3 Residue graphs as representations of protein complexes

A protein complex can be represented as a graph $G = (V, E)$ in which each node $i \in V$ corresponds to a residue and each edge $(i, j) \in E$ corresponds to a residue pair connected by a chosen geometric or interaction criterion. This representation is attractive because it preserves local structural relationships while remaining much smaller than an atom-level graph. It also aligns naturally with residue-level descriptors such as solvent exposure, buried surface area, residue depth, or physicochemical identity. DeepRank2 provides

a mature framework for constructing such residue graphs and attaching biophysically motivated node and edge features [2].

In this thesis, the graph representation serves two purposes. First, it organizes the static structural information extracted from a prepared protein complex. Second, it provides a scaffold on which molecular-dynamics-derived summaries can be added as node and edge attributes. The graph itself is therefore static in topology, but some of its attributes summarize time-dependent behaviour. This is computationally convenient, but it also imposes a modelling assumption: dynamics are represented as statistics on a fixed graph rather than as a genuinely time-varying graph process.

Two graph views are important throughout the thesis. The *full* view retains the whole complex graph and therefore allows information to flow through the entire structure. The *interface* view retains only a local patch around the predicted binding region. Interface-focused modelling is intuitive because binding occurs at an interface, but it can also discard long-range context such as distal electrostatics, packing constraints, or broader chain geometry. Comparing these two views is therefore a scientifically meaningful design choice rather than a purely computational convenience.

3.4 Graph neural networks and message passing

Graph neural networks (GNNs) learn node and graph representations by iteratively exchanging information along graph edges. In a generic message-passing layer, a node embedding $\mathbf{h}_i^{(\ell)}$ at layer ℓ is updated using its own current state and aggregated messages from neighbouring nodes:

$$\mathbf{h}_i^{(\ell+1)} = \phi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \sum_{j \in \mathcal{N}(i)} \psi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ij} \right) \right). \tag{3.5}$$

Here $\mathbf{e}_{ij}$ denotes the edge features on (i, j), $\psi^{(\ell)}$ computes a message, and $\phi^{(\ell)}$ updates the node state. After several such layers, the graph can be summarized by a readout operation

$$\mathbf{g} = \rho \left(\{ \mathbf{h}_i^{(L)} : i \in V \} \right), \tag{3.6}$$

where ρ is a permutation-invariant pooling operator such as mean, max, or their concatenation.

The specific convolution used in this thesis is GINE, an edge-aware extension of the graph isomorphism network family [3, 4]. GIN-style models are attractive because they are expressive relative to simpler neighbourhood-averaging architectures, and the GINE variant allows edge attributes to contribute explicitly to message passing. This is important here because both static and dynamic edge features are part of the representation. Once

a graph-level embedding has been obtained, it can either be used directly for supervised regression or passed into a latent-variable model such as a variational autoencoder.

3.5 Variational autoencoders and masked reconstruction

A variational autoencoder (VAE) is a latent-variable model that combines an encoder, which maps an input to a distribution over latent variables, and a decoder, which reconstructs the input from a sample of the latent variable [17]. In the standard formulation, the encoder approximates a posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$, the decoder defines a likelihood model $p_\theta(\mathbf{x} \mid \mathbf{z})$, and training maximizes a variational lower bound,

$$\log p_\theta(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p_\theta(\mathbf{x} \mid \mathbf{z})\right] - D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z}))\,. \tag{3.7}$$

The reconstruction term encourages the latent variable to retain useful information about the input, while the Kullback–Leibler term encourages the latent distribution to remain close to a simple prior, usually a standard normal distribution.

In this thesis, the VAE is not used as a classical generative model of entire protein graphs. Instead, it is used as a structured representation learner. The encoder maps a residue graph to a graph-level latent mean and variance, and the decoder is trained to reconstruct selected masked node and edge features. This makes the objective closer in spirit to masked self-supervised learning than to unconditional generation. An additional semi-supervised affinity head can be attached to the latent mean so that the same latent space is encouraged to support both feature reconstruction and affinity prediction. The appeal of this design is that it may learn a compact latent representation that captures both structure and affinity-relevant variation, while still allowing a simple downstream regressor to test how informative the learned representation really is.

3.6 Why repeated outer resampling matters

Model selection on a single train/validation/test split is fragile, especially when many closely related configurations are compared. A model can appear superior simply because its inductive biases happened to align well with one particular split. Repeated outer resampling is a practical way to reduce this fragility. Instead of evaluating a configuration on only one held-out test set, the dataset is partitioned repeatedly into new train/validation/test subsets, the model is retrained from scratch each time, and performance is averaged across the resulting held-out fits.

This design is particularly important in the present thesis because the goal is not

merely to find a single good model, but to compare design choices such as full versus interface graphs, S versus SD inputs, variational versus direct supervised objectives, and different reconstruction target policies. A repeated outer design does not eliminate all uncertainty, but it does make it much harder for a single lucky split to produce a misleading scientific conclusion. The confirmatory matrix in Chapter 5 should be understood in this light: it is primarily a robustness study of model-design decisions.

Chapter 4

Methods

4.1 Dataset curation and affinity labels

All experiments were based on the PPB-Affinity resource, a large protein–protein binding affinity dataset assembled for machine-learning applications in macromolecular drug discovery [1]. The active dataset used in this study was a locally curated broad subset that retained all major PPB-Affinity subgroups while enforcing a single annotated ligand–receptor pairing per complex. Complexes were retained only if a Protein Data Bank (PDB) structure, an interpretable ligand and receptor chain assignment, and an affinity measurement sufficient to derive a binding free energy were all available. Mutant complexes were excluded so that the final learning problem remained a wild-type structure-to-affinity mapping. Duplicate entries were removed by requiring both a unique PDB identifier and a unique ordered ligand/receptor chain-group definition. The resulting curated CSV contained 2818 complexes.

Each complex was assigned a canonical identifier of the form `PDB__ligandChains__receptorChain` Experimental affinity was represented as the standard Gibbs free energy of binding, ΔG, in kcal mol^{-1}. When a free energy value was not already present in the source table, it was computed from the reported dissociation constant K_D using

$$\Delta G = RT \ln K_D, \tag{4.1}$$

where $R = 0.00198720425864083$ kcal mol^{-1} K^{-1} and T is the experimental temperature in kelvin. If a complex-specific temperature was not available, $T = 298.15$ K was used. This conversion was applied uniformly during dataset normalization so that all downstream models were trained against a single continuous thermodynamic target.

4.2 Molecular dynamics system preparation

Protein complexes were prepared from their PDB coordinate files using PDBFixer [21] and the OpenMM application layer [7]. The annotated ligand and receptor chains were first extracted from the source complex. Missing residues were detected during preprocessing but were not rebuilt; this choice preserved the experimentally deposited backbone definition rather than introducing modeled insertions. Nonstandard residues were converted when PDBFixer provided a standard mapping, heterogens were removed, missing heavy atoms were added, and hydrogens were assigned before solvation. Only protein residues were retained in the final simulation system prior to solvent addition.

Prepared complexes were solvated explicitly with a cubic water shell extending 1.0 nm beyond the solute and with an ionic strength of 0.15 M. The force-field stack used throughout the active pipeline was `amber14-all.xml` for the protein and `amber14/tip3pfb.xml` for the solvent, corresponding to the Amber14 protein force field and the TIP3P-FB water model [22, 23]. Protonation-state preparation used the per-complex experimental pH when available and otherwise defaulted to pH 7.0. In the final retained analysis set, experimental pH metadata were not available and the default pH was therefore used uniformly.

4.3 Molecular dynamics protocol

All simulations were carried out in OpenMM [7]. The molecular mechanics system was built with particle-mesh Ewald electrostatics, a 1.0 nm real-space nonbonded cutoff, constraints on bonds involving hydrogen, rigid water, and an Ewald error tolerance of 10^{-4}. Pressure was controlled at 1 atm using a Monte Carlo barostat and temperature was controlled with a Langevin middle integrator using a friction coefficient of 1 ps^{-1} and a 2 fs time step. Each system was energy-minimized for up to 1000 iterations, equilibrated for 100,000 steps, and then propagated for a 500,000-step production run. At a 2 fs step size, these settings correspond to 0.2 ns of equilibration followed by 1.0 ns of additional simulation.

Trajectory and state reporters were written every 5000 integration steps, corresponding to 10 ps between saved frames. Two NetCDF trajectories were written for each complex: a full-system trajectory containing protein, solvent, and ions, and a second protein-only trajectory written in the same coordinate frame. Because the reporters were attached before equilibration began, all saved frames from both the equilibration and production phases were available to downstream feature extraction. The stored outputs for each complex therefore comprised 120 saved frames over a total simulated duration of 1.2 ns. Complex-specific temperatures from the source dataset were used when available; otherwise a default temperature of 298.15 K was used.

4.4 Residue-graph construction

Residue-level graphs were generated with DeepRank2 [2]. To avoid restricting the graph to a predefined interface only, the active preparation pipeline used DeepRank2 in a full-complex query mode with an effectively unbounded influence radius of 10^6 Å, which ensured that all protein residues in the prepared topology were represented in the resulting graph. DeepRank2 feature modules `components`, `contact`, `exposure`, and `surfacearea` were applied to the protein-only topology written from the molecular dynamics preparation stage. This yielded per-residue structural descriptors including buried surface area, solvent-accessible surface area, residue depth, and half-sphere exposure features, together with pairwise edge descriptors including residue distance, same-chain identity, electrostatic interaction, and van der Waals interaction.

These DeepRank2 features were augmented with two additional categories of node attributes. First, residue identity was encoded using a 20-way amino-acid one-hot representation together with coarse physicochemical indicators and scalar descriptors of charge, hydropathy, mass, aromaticity, and hydrogen-bonding tendency. Second, a local structural-context count was computed for each residue on the first saved protein frame by counting residues within 8 Å on the same chain and on the partner chain. These context counts were concatenated with the DeepRank2 node features and treated as static inputs in all downstream models.

4.5 Molecular dynamics-derived node and edge features

Dynamic graph attributes were computed from the saved trajectories using MDTraj [8]. For each complex, at most 120 frames were used during feature extraction; given the reporting schedule above, this corresponded to using all saved frames from the 1.2 ns trajectory. Dynamic node features comprised hydration, flexibility, backbone torsional state, and inter-chain force-proxy summaries. Dynamic edge features comprised contact frequency and distance statistics evaluated on the fixed edge set of the static graph.

Hydration was quantified from the full solvated trajectory. For each residue and frame, all unique water oxygen residues within 5 Å of any heavy atom of that residue were counted. The stored features were $\log(1+\bar{w}_i)$, where $\bar{w}_i$ is the mean water count for residue i across frames, and the standard deviation of the framewise water counts. Conformational flexibility was represented by the Cα root-mean-square fluctuation (RMSF) computed after least-squares superposition of the trajectory onto itself using Cα atoms. Backbone torsional state was summarized by computing ϕ and ψ angles for all residues with defined dihedrals and storing the frame-averaged values of $\sin\phi$, $\cos\phi$, $\sin\psi$, and $\cos\psi$.

Dynamic edge features were computed only for residue pairs connected by an existing

static graph edge. For each such pair, MDTraj's closest-heavy-atom distance was evaluated in every saved frame. The dynamic contact frequency for edge (i, j) was then defined as the fraction of frames in which the closest-heavy distance was at most 8 Å. When distance statistics were enabled, the mean and standard deviation of the closest-heavy distance across frames were also stored as dynamic edge attributes. This representation should therefore be interpreted as a static graph with time-aggregated edge annotations rather than as a fully time-varying graph.

4.6 Approximate inter-chain force features

In addition to hydration and conformational descriptors, each residue was assigned an approximate inter-chain nonbonded force summary. These features were derived from the protein-only trajectory and a reconstructed OpenMM analysis system built from the saved protein topology using the same force-field files as the original simulation. To isolate direct ligand–receptor interactions, the original OpenMM `NonbondedForce` parameters were copied into two separate `CustomNonbondedForce` objects, one representing a Coulomb term and one representing a Lennard–Jones term, and each custom force was restricted to ligand-atom versus receptor-atom interaction groups using OpenMM interaction-group semantics [7]. The custom analysis system used periodic cutoffs and excluded the original exception pairs; long-range correction was disabled for the custom forces.

For each saved frame, per-atom forces from the Coulomb and Lennard–Jones custom terms were accumulated over all atoms belonging to the same residue. The Euclidean norm of the resulting residue-level force vector was then taken, and the mean and standard deviation of these norms across frames were stored separately for the Coulomb and Lennard–Jones channels. Because these quantities were computed on reconstructed protein-only systems with direct ligand–receptor interaction groups rather than extracted directly from the full solvated PME simulation context, they should be interpreted as approximate inter-chain nonbonded force proxies rather than exact total molecular dynamics forces.

4.7 Graph views and matched analysis subset

Two graph views were evaluated. The *full* view retained the complete residue graph for each complex. The *interface* view retained only a fixed-size local patch intended to emphasize the interaction region while keeping the input size approximately constant across complexes. For the final confirmatory experiments, the interface policy was `md_closest_pair_patch`. Under this policy, mapped Cα coordinates from the molecular dynamics trajectory were used to identify the ligand residue and receptor residue with

the minimum inter-chain Cα distance. These two residues formed the interface core, and the final node set was obtained by expanding locally around this core under a hard cap of 128 residues. The corresponding edge set was induced from the original graph.

To support a fair comparison between the full and interface representations, both views were restricted to the same retained complex subset. Specifically, the interface materialization stage was run in a matched-subset mode so that any complex without a valid interface patch was removed from both views. The final confirmatory analysis therefore used 1437 matched complexes, each represented once as a full graph and once as an interface graph derived from the same prepared record.

4.8 Feature modes and preprocessing

Two input modes were considered. In the *static* mode, denoted S, model inputs consisted only of the static node and edge features described above. In the *static-plus-dynamic* mode, denoted SD, the input feature vector was augmented with the dynamic node features, torsion summaries, force-proxy summaries, and dynamic edge features. The edge feature `same_chain` was retained as an input feature in both modes but was excluded from reconstruction targets because it is topologically trivial once the graph has been constructed.

Feature scaling was fit exclusively on the training split of each run. For every numeric feature, the training distribution was first clipped to its 0.5th and 99.5th percentiles, then standardized to zero mean and unit variance, and finally clipped again to the interval $[-8, 8]$. The same transformation was then applied unchanged to the corresponding validation and test splits. This preprocessing was identical for the GraphVAE and the direct supervised baseline, ensuring that any performance differences were attributable to model design rather than to differences in feature normalization.

4.9 Masked graph variational autoencoder

The unsupervised and semi-supervised graph representation models were implemented as masked graph variational autoencoders using PyTorch Geometric [5]. Both node and edge inputs were projected linearly to a hidden dimension of 128 and passed through three graph isomorphism network with edge features (GINE) convolution layers [4]. Graph-level embeddings were obtained by concatenating global mean pooling and global max pooling. Two linear heads then produced the latent mean vector μ and log-variance vector $\log \sigma^2$. The latent dimensionality was varied over $d_z \in \{8, 16, 32\}$ in the final model matrix.

The decoder reconstructed selected feature subsets rather than the full input tensor. For each batch, target node and edge entries were independently masked with probability 0.30. Decoder inputs then consisted of the sampled latent vector together with

the unmasked graph inputs, the partially masked target features, and explicit binary mask indicators. Three target policies were used in the confirmatory experiments. The `shared_static` policy reconstructed static residue descriptors and the nontrivial static edge distance feature. The `sd_dynamic_all` policy reconstructed only dynamic node and dynamic edge attributes. The `sd_static_plus_dynamic_all` policy reconstructed the union of static and dynamic targets. Only the `shared_static` policy was used for S runs, whereas all three policies were used for SD runs.

For a batch of graphs, the total GraphVAE objective was

$$\mathcal{L} = \mathcal{L}_{\text{node}}^{\text{recon}} + \mathcal{L}_{\text{edge}}^{\text{recon}} + \beta \mathcal{L}_{\text{KL}} + \lambda_{\text{corr}} \mathcal{L}_{\text{corr}} + \lambda_{\text{aff}} \mathcal{L}_{\text{aff}}, \tag{4.2}$$

where the node and edge reconstruction terms were masked mean-squared errors over the selected target entries, the Kullback–Leibler term was

$$\mathcal{L}_{\text{KL}} = \frac{1}{B d_z} \sum_{b=1}^{B} \sum_{k=1}^{d_z} \left[-\frac{1}{2} \left(1 + \log \sigma_{bk}^2 - \mu_{bk}^2 - \sigma_{bk}^2 \right) \right], \tag{4.3}$$

and $\mathcal{L}_{\text{corr}}$ penalized the squared off-diagonal elements of the latent covariance matrix to discourage redundant latent dimensions. In semi-supervised runs, $\mathcal{L}_{\text{aff}}$ was a mean-squared error between the normalized affinity prediction from a latent MLP head and the normalized experimental target. The exact confirmatory settings were $\lambda_{\text{corr}} = 0.01$, $\lambda_{\text{aff}} = 1.0$ for semi-supervised runs and 0 otherwise, $\beta_{\text{start}} = 0$, $\beta_{\text{end}} = 1$, and a linear β warm-up over the first 30% of training epochs. With a maximum of 120 epochs, the KL weight therefore increased linearly over the first 36 epochs and remained at 1.0 thereafter.

All GraphVAE models were optimized with AdamW using a learning rate of 10^{-4}, weight decay of 10^{-5}, batch size 16, and gradient clipping at a global norm of 5. Training used a maximum of 120 epochs with early stopping patience of 20 epochs. Checkpoint selection was based on the validation reconstruction loss, plus the weighted validation affinity loss in semi-supervised runs. The KL and correlation terms regularized training but did not directly determine which checkpoint was retained as the best model.

4.10 Direct supervised baseline

To separate the value of the graph representation from the value of the variational objective, each graph view and feature mode was also evaluated with a direct supervised baseline. This model used the same input feature definitions, the same hidden dimension of 128, the same three GINE layers, and the same mean-plus-max graph pooling strategy as the GraphVAE encoder. The difference was that the pooled graph embedding was passed directly to a multilayer perceptron regressor trained to predict ΔG with a mean-squared-error loss. The supervised baseline was optimized with AdamW using the same

learning rate, weight decay, batch size, and maximum epoch count as the GraphVAE runs. Early stopping was based on validation mean squared error, with patience fixed to 20 epochs for the final confirmatory matrix.

4.11 Latent-space regression and final VAE evaluation

Final GraphVAE performance was not reported solely from the neural affinity head. Instead, each retained checkpoint was treated as a representation learner, and its graph-level latent mean vectors were exported for the train, validation, and test splits. A ridge regressor was then fit on the training latent vectors only,

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \|\mathbf{y}_{\text{train}} - \mathbf{X}_{\text{train}}\mathbf{w}\|_2^2 + \alpha\|\mathbf{w}\|_2^2, \tag{4.4}$$

and evaluated on the held-out validation and test splits. The regularization parameter was selected with `RidgeCV` from scikit-learn [24] over the grid

$$\alpha \in \{10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}, 3\times10^{-2}, 10^{-1}, 3\times10^{-1}, 1, 3, 10, 30, 100\},$$

using five-fold inner cross-validation on the training split only. No additional repeated cross-validation was performed inside this probe during the final resampling study. The headline GraphVAE metrics reported in the confirmatory results therefore quantify the predictive quality of the learned latent representation under a simple regularized linear readout, rather than the performance of the end-to-end neural affinity head alone.

4.12 Repeated outer resampling and model matrix

The confirmatory analysis used repeated outer resampling on the matched 1437-complex set. Unique complex identifiers, rather than individual graphs or trajectory frames, were the unit of resampling. For each of two repeats, complexes were randomly permuted with seed 2026 and partitioned into five outer folds. One fold was held out as the test set and the remaining four folds were split into training and validation subsets using a validation fraction of 0.1875 of the train-plus-validation pool, which yields the intended overall 70/15/15 proportion within each outer fit. In practice, each fit therefore contained approximately 934 training complexes, 215–216 validation complexes, and 287–288 test complexes. All preprocessing, model fitting, checkpoint selection, and ridge fitting were repeated independently within each outer split.

The final matrix contained 52 configurations. Forty-eight of these were GraphVAE runs spanning two graph views (full and interface), two input modes (S and SD), two

supervision settings (unsupervised and semi-supervised), and three latent dimensionalities (8, 16, and 32). For SD runs, three target policies were evaluated; for S runs, only `shared_static` was evaluated because the dynamic target policies are not distinct when dynamic inputs are absent. Four additional runs were direct supervised baselines corresponding to the Cartesian product of graph view and feature mode. All models used the same hidden dimension (128), number of GINE layers (3), learning rate (10^{-4}), weight decay (10^{-5}), batch size (16), maximum epochs (120), and early-stopping patience (20) in the final confirmatory matrix.

4.13 Evaluation metrics

For every outer fit, predictive performance was summarized on the held-out test set using root-mean-square error (RMSE), coefficient of determination (R^2), and Pearson correlation coefficient. For n test complexes with observed affinities y_i and predictions $\hat{y}_i$, these were computed as

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{4.5}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{4.6}$$

and

$$r = \frac{\sum_{i=1}^{n}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}\sqrt{\sum_{i=1}^{n}(\hat{y}_i - \bar{\hat{y}})^2}}. \tag{4.7}$$

For the final report, metrics were aggregated across the 10 outer fits per configuration and summarized by their mean, standard deviation, minimum, and maximum. This repeated held-out evaluation formed the basis of all factor-level conclusions about graph view, feature mode, supervision, target policy, and latent dimensionality.

Chapter 5

Results

This chapter reports the final confirmatory evaluation described in Chapter 4. The analysis was restricted to the matched 1437-complex subset for which both the full and interface graph views were available after molecular-dynamics processing, force-feature construction, and interface materialization. The final model matrix comprised 52 configurations: 48 GraphVAE-based representation models and 4 direct supervised baselines. Each configuration was evaluated across 10 outer held-out fits obtained from 5 folds and 2 repeats. If $m_{c,t}$ denotes a metric for configuration c on outer fit $t \in \{1, \ldots, 10\}$, the reported configuration-level summary is

$$\bar{m}_c = \frac{1}{10} \sum_{t=1}^{10} m_{c,t}. \tag{5.1}$$

All rankings and factor comparisons below are based on these configuration-level means. For paired factor comparisons, descriptive 95% intervals over paired configuration deltas are reported as $\bar{\Delta} \pm 1.96\, s_{\Delta}/\sqrt{n}$, where n is the number of paired configurations. These intervals are included to show the stability of the direction and magnitude of the observed effect; they are not formal hypothesis tests.

5.1 Headline model performance

Table 5.1 lists the strongest configurations from the final matrix. The best overall model was the direct supervised baseline on the full static graph, `full_base_S`, with mean test RMSE 2.3379, mean test Pearson correlation 0.5064, and mean test R^2 0.2146. The best GraphVAE configuration was `full_S_semi_shared_static_z32`, with mean test RMSE 2.4292, mean test Pearson correlation 0.4649, and mean test R^2 0.1537. The best static-plus-dynamic GraphVAE configuration was `full_SD_semi_shared_static_z8`, with mean test RMSE 2.4364 and mean test Pearson correlation 0.4749. The best interface-only models were materially weaker than their full-graph counterparts. The strongest interface baseline, `iface_base_S`, achieved mean test RMSE 2.5782, while the best interface

Configuration	Test RMSE	Test r	Test R^2	Test−Train RMSE gap
`full_base_S`	2.3379	0.5064	0.2146	0.8021
`full_base_SD`	2.4073	0.4802	0.1685	0.8414
`full_S_semi_shared_static_z32`	2.4292	0.4649	0.1537	0.7451
`full_SD_semi_shared_static_z8`	2.4364	0.4749	0.1478	1.1303
`iface_base_S`	2.5782	0.3451	0.0470	0.8090
`iface_S_semi_shared_static_z8`	2.6041	0.3402	0.0282	0.8018
`iface_SD_semi_sd_static_plus_dynamic_z32`	2.6238	0.3252	0.0146	0.7597

Table 5.1: Selected headline models from the final 52-configuration confirmatory matrix. Metrics are means over 10 outer held-out fits on the matched 1437-complex set. For GraphVAE models, the reported predictive metrics come from the downstream ridge probe on latent means rather than solely from the neural affinity head.

GraphVAE, `iface_S_semi_shared_static_z8`, achieved mean test RMSE 2.6041.

The best direct supervised baseline outperformed the best GraphVAE by 0.0913 RMSE units and by 0.0415 Pearson-correlation units on average. Within the SD family, the best model was again a full-graph method rather than an interface method: `full_base_SD` reached mean test RMSE 2.4073 and mean test $r = 0.4802$, while the best SD GraphVAE reached mean test RMSE 2.4364 and mean test $r = 0.4749$. The confirmatory matrix therefore did not support a claim that the variational representation-learning objective outperformed a direct supervised graph regressor on the present dataset and feature set.

5.2 Effect of graph view: full versus interface

The clearest factor effect in the matrix was the advantage of the full graph view over the interface graph view. There were 24 matched GraphVAE configuration pairs differing only in whether the model was trained on `full` or `interface` records. In all 24 of these paired comparisons, the full graph achieved a lower mean test RMSE than the corresponding interface graph. Defining the paired RMSE penalty for the interface representation as

$$\Delta_{\mathrm{RMSE}}^{\mathrm{iface-full}} = \bar{\mathrm{RMSE}}_{\mathrm{iface}} - \bar{\mathrm{RMSE}}_{\mathrm{full}}, \tag{5.2}$$

the mean paired penalty was 0.1005 RMSE units, with a descriptive 95% interval of [0.0653, 0.1356]. If the corresponding Pearson-correlation advantage is defined as

$$\Delta_{r}^{\mathrm{full-iface}} = \bar{r}_{\mathrm{full}} - \bar{r}_{\mathrm{iface}}, \tag{5.3}$$

the mean full-graph advantage was 0.0993, with a descriptive 95% interval of [0.0847, 0.1138].

The same direction held at the family level. Averaging over all 24 full-graph GraphVAE configurations gave mean test RMSE 2.5494 and mean test Pearson correlation 0.2883. Averaging over all 24 interface GraphVAE configurations gave mean test RMSE 2.6498

and mean test Pearson correlation 0.1890. The direct supervised baselines showed the same pattern. The best full baseline, `full_base_S`, achieved mean test RMSE 2.3379, whereas the best interface baseline, `iface_base_S`, achieved mean test RMSE 2.5782. This confirms that the superiority of the full representation was not specific to the variational model family.

These results are sufficiently uniform that the main view-level conclusion is not sensitive to model-selection details. On the present matched subset, with the present residue graph definitions and interface-patch construction, the full-graph representation was consistently more predictive than the interface-only representation.

5.3 Effect of dynamic features: S versus SD

The second main question was whether augmenting the static graph with molecular-dynamics-derived features improved predictive performance. The cleanest comparison for this purpose is between matched S and SD configurations under the `shared_static` reconstruction target, because both models are then trained under the same target policy and differ only in whether the dynamic channels are included as inputs. There were 12 such matched GraphVAE pairs in the final matrix. In 11 of the 12 pairs, the S model achieved a lower mean test RMSE than the corresponding SD model.

Defining the paired dynamic-feature penalty as

$$\Delta_{\mathrm{RMSE}}^{\mathrm{SD-S}} = \mathrm{R\bar{M}SE}_{\mathrm{SD}} - \mathrm{R\bar{M}SE}_{\mathrm{S}}, \tag{5.4}$$

the mean penalty was 0.0254 RMSE units, with a descriptive 95% interval of [0.0134, 0.0375]. The corresponding Pearson-correlation difference,

$$\Delta_{r}^{\mathrm{S-SD}} = \bar{r}_{\mathrm{S}} - \bar{r}_{\mathrm{SD}}, \tag{5.5}$$

favored the static-only representation by 0.0215 on average, with a descriptive 95% interval of [0.0043, 0.0387]. The direct supervised baselines were consistent with the same conclusion: `full_base_SD` was worse than `full_base_S` by 0.0694 RMSE units, and `iface_base_SD` was not better than `iface_base_S`.

This effect was small rather than catastrophic. The best SD GraphVAE, `full_SD_semi_shared_st` was only 0.0072 RMSE units worse than the best S GraphVAE, `full_S_semi_shared_static_z32`, and it had a slightly higher test Pearson correlation (0.4749 versus 0.4649). The problem is therefore not that the dynamic features always destroy performance. The problem is that, across the full confirmatory matrix, the dynamic augmentation does not yield a robust and repeatable generalization gain over the static representation.

5.4 Effect of supervision

The influence of semi-supervision depended strongly on graph view. Within the full graph view, semi-supervision helped consistently. For the static mode, all 3 of the 3 matched full-graph pairs favored semi-supervision over the corresponding unsupervised model, with a mean test-RMSE gain of 0.1640. For the static-plus-dynamic mode, all 9 of the 9 matched full-graph pairs favored semi-supervision, with a mean test-RMSE gain of 0.1604. The strongest full-graph models therefore all came from the semi-supervised branch of the matrix.

The interface results were weaker and less coherent. In the static mode, semi-supervision still helped in all 3 of the 3 matched interface pairs, but the mean gain was only 0.0290 RMSE units. In the dynamic mode, semi-supervision did not produce a robust benefit. Across the 9 matched interface SD pairs, the mean semi-supervised gain was -0.0081 RMSE units and only 3 of the 9 pairs favored the semi-supervised model. The interpretation is therefore not simply that supervision helps universally. Rather, supervision helps when the graph representation contains sufficient predictive structure, which appears to be true for the full graphs and only marginally true for the interface graphs.

5.5 Effect of reconstruction target policy

Within the SD GraphVAE branch, the confirmatory matrix also tested whether the decoder should reconstruct only static targets, only dynamic targets, or both. The clearest regime in which to examine this question is the full-graph semi-supervised SD family, because this was the only dynamic branch that approached competitive held-out performance. Averaging over the three latent dimensionalities gave mean test RMSE 2.4434 for `shared_static`, 2.4795 for `sd_static_plus_dynamic_all`, and 2.5150 for `sd_dynamic_all`. The corresponding mean test Pearson correlations were 0.4682, 0.4335, and 0.4221, respectively.

Pairwise comparisons by latent dimensionality showed that reconstructing only dynamic targets was consistently worse than reconstructing the shared static targets. Across the three matched latent-dimensionality pairs, the mean RMSE penalty of `sd_dynamic_all` relative to `shared_static` was 0.0716, with a descriptive 95% interval of [0.0305, 0.1127]. The mean Pearson-correlation loss was 0.0461, with a descriptive 95% interval of [0.0158, 0.0764]. The `sd_static_plus_dynamic_all` target policy was intermediate. Its mean RMSE penalty relative to `shared_static` was 0.0360, although this comparison was less stable across latent dimensionality because only three matched pairs were available. The corresponding mean Pearson-correlation loss was still positive at 0.0347.

These results indicate that, in the only dynamic regime that was even moderately competitive, asking the decoder to reconstruct dynamic targets did not improve held-

out predictive quality. The strongest dynamic models were obtained when the dynamic channels were present as inputs but the reconstruction objective remained focused on the shared static targets.

5.6 Overfitting and train–validation–test behaviour

The confirmatory matrix also yielded a clear generalization story. To quantify overfitting, define the train–test RMSE gap and train–test correlation gap for configuration c as

$$g_{\mathrm{RMSE}}(c) = \bar{\mathrm{RMSE}}_{\mathrm{test},c} - \bar{\mathrm{RMSE}}_{\mathrm{train},c}, \tag{5.6}$$

and

$$g_r(c) = \bar{r}_{\mathrm{train},c} - \bar{r}_{\mathrm{test},c}. \tag{5.7}$$

Large positive values of either quantity indicate stronger train-to-test divergence.

The unsupervised GraphVAE families were characterized primarily by underfitting rather than by severe overfitting. For example, the mean train, validation, and test Pearson correlations for the full unsupervised static family were 0.2486, 0.1897, and 0.2055, respectively, with a mean train–test RMSE gap of only 0.0276. The corresponding full unsupervised dynamic family had mean train, validation, and test correlations of 0.1809, 0.0967, and 0.1074. These small train–test gaps do not indicate strong robustness; they indicate that the unsupervised models did not learn much affinity signal even on their training sets.

The overfitting story emerges in the semi-supervised families. The full semi-supervised static family had mean train, validation, and test Pearson correlations of 0.7494, 0.4666, and 0.4551, respectively, with a mean train–test RMSE gap of 0.6984. This is a moderate and credible generalization gap. The full semi-supervised dynamic family fit the training data substantially more strongly, with mean train correlation 0.8218, but it did not improve held-out performance; its mean test correlation was only 0.4413 and its mean train–test RMSE gap increased to 1.0049. The interface semi-supervised dynamic family exhibited the strongest divergence of all major families, with mean train, validation, and test correlations of 0.7685, 0.3231, and 0.3189, respectively, and a mean train–test RMSE gap of 1.0140.

Table 5.2 summarizes these family-level patterns. The best direct supervised baseline, `full_base_S`, combined the highest held-out performance with a more controlled generalization gap than the semi-supervised dynamic families. Its mean train, validation, and test correlations were 0.8225, 0.5337, and 0.5064, respectively, and its mean train–test RMSE gap was 0.8021. This is still a real gap, but it is materially smaller than the dynamic semi-supervised VAE families while producing better held-out accuracy.

The worst individual overfitting case in the semi-supervised GraphVAE matrix was

Family	Train r	Val r	Test r	g_r	Train RMSE	Test RMSE
Full baseline S	0.8225	0.5337	0.5064	0.3161	1.5358	2.3379
Full baseline SD	0.8077	0.5191	0.4802	0.3275	1.5659	2.4073
Full semi-supervised VAE S	0.7494	0.4666	0.4551	0.2942	1.7385	2.4369
Full semi-supervised VAE SD	0.8218	0.4580	0.4413	0.3805	1.4744	2.4793
Interface semi-supervised VAE S	0.7083	0.3356	0.3361	0.3722	1.8467	2.6112
Interface semi-supervised VAE SD	0.7685	0.3231	0.3189	0.4496	1.6479	2.6620

Table 5.2: Family-level train/validation/test behaviour. Each row is the arithmetic mean over all configurations in that family, using the configuration-level means from the 10 outer held-out fits. Here $g_r = \bar{r}_{\text{train}} - \bar{r}_{\text{test}}$.

`iface_SD_semi_shared_static_z32`. This model reached mean train Pearson correlation 0.8213 but only mean test Pearson correlation 0.3185, giving a train–test correlation gap of 0.5028. Its mean train RMSE was 1.4239, while its mean test RMSE was 2.6705. This specific result reinforces the broader pattern seen at the family level: adding dynamic channels, especially on the interface representation, increased training-set fit more than it increased robust held-out predictive signal.

5.7 Summary of empirical conclusions

The confirmatory matrix supports four main empirical conclusions. First, the full graph view was consistently and substantially stronger than the interface graph view for the present problem setup. Second, the current molecular-dynamics-derived feature construction did not provide a robust generalization advantage over the static representation. Third, within the dynamic GraphVAE branch, reconstruction objectives focused on dynamic targets were not helpful and were usually harmful. Fourth, the strongest overall model was the direct supervised baseline on the full static graph, not the variational model.

These conclusions are stronger than the corresponding conclusions from earlier single-split experiments because they are based on repeated outer held-out evaluation over 52 configurations and 520 total fits. At the same time, the results remain specific to the present implementation choices: short explicit-solvent trajectories, static residue graphs with time-aggregated annotations, a 128-node interface patch, and approximate protein-only inter-chain force proxies. The negative result for the current dynamic encoding should therefore be interpreted as a result about this particular representation strategy rather than as a universal statement that protein dynamics are irrelevant for affinity prediction.

Chapter 6

Discussion and Conclusions

6.1 Principal findings

The final confirmatory matrix leads to one strong positive conclusion and one strong negative conclusion. The positive conclusion is that the full residue graph is consistently more informative than the interface-only graph for the present problem setup. The negative conclusion is that the present molecular-dynamics-derived feature encoding does not provide a robust held-out advantage over the static representation. These conclusions are not based on one favourable split, but on a 52-configuration matrix evaluated over 520 total outer fits on a matched 1437-complex subset.

The strongest overall model was the direct supervised baseline on the full static graph, `full_base_S`, with mean test RMSE 2.3379 and mean test Pearson correlation 0.5064. The best variational model, `full_S_semi_shared_static_z32`, was weaker on both metrics. The best dynamic variational model, `full_SD_semi_shared_static_z8`, came close to the best static variational model, but did not surpass the full static baseline and did not establish a stable matrix-wide advantage for the dynamic feature set. These are not the most optimistic results one might have hoped for at the start of the project, but they are precise, interpretable, and scientifically useful.

6.2 Why the full graph outperformed the interface graph

The superiority of the full graph is one of the most stable results in the thesis. Every matched variational comparison favored the full representation, and the same direction held for the supervised baselines. The most plausible explanation is that the current interface construction discards too much information. In the final retained set, full graphs contained on average 495.9 nodes and 10 697.2 edges, whereas interface graphs contained on average 127.3 nodes and 2106.3 edges. More importantly, 1395 of the 1437 interface

graphs, or 97.1%, reached the hard cap of 128 nodes. This means that the interface representation was not merely focusing attention on a small local region; in most cases it was being actively truncated at the patch limit.

That truncation matters because affinity is not a purely local property of the closest contact region. Long-range electrostatics, secondary-shell packing, broader chain geometry, and network effects propagated through the graph can all influence a graph-level predictor. The full graph preserves this context, whereas the interface graph imposes an aggressive bottleneck. The interface patch is also defined operationally, in the final experiments, around the closest inter-chain Cα pair from the molecular-dynamics-mapped coordinates. This is a sensible local anchor, but it is still a heuristic. If the most predictive residues are not fully captured by that local expansion, the resulting graph will be intrinsically information-limited.

The practical conclusion is therefore not that interface modelling is inherently weak. It is that the specific 128-node interface patch used here is too restrictive to compete with the full graph on the matched PPB-Affinity subset. Any future claim that interface-only graphs should dominate would need either a richer interface definition or evidence from a different and less saturating patching strategy.

6.3 Why the present dynamic features did not help

The dynamic result requires more careful interpretation. It would be too strong to conclude that dynamics are unimportant for protein–protein affinity. What the results support is a narrower statement: the present dynamic representation does not add a robust held-out benefit over the static graph. Several aspects of the implementation help explain why.

First, the trajectory budget is short. Each complex contributes 1.2 ns of saved trajectory, of which 0.2 ns are equilibration frames and 1.0 ns are additional production frames. This is enough to estimate local fluctuations and contact persistence at a coarse level, but it is not long enough to explore major conformational rearrangements for most protein complexes. Second, the trajectory is summarized very aggressively. Hydration is reduced to a per-residue mean and standard deviation of nearby water counts, flexibility is reduced to Cα RMSF, torsional state is reduced to mean sin / cos values, and edge dynamics are reduced to contact frequency and distance moments on a fixed pre-existing edge set. Temporal order, slow state switching, and changing graph topology are all discarded.

Third, the force features are only approximate proxies. They are based on reconstructed protein-only analysis systems and direct ligand–receptor Coulomb and Lennard–Jones interaction groups, not on exact force decomposition from the full solvated PME simulation context. As a consequence, they are physically interpretable but only at the

level of coarse direct inter-chain nonbonded magnitude summaries. Fourth, the simulation metadata are imperfect. Missing residues are not rebuilt, non-protein components are removed before simulation, trajectories include equilibration frames, and pH defaults to 7.0 because pH metadata are absent from the dataset. Experimental temperature is also frequently unavailable and therefore defaults to 298.15 K in many cases. None of these choices invalidates the pipeline, but together they make it easier for the dynamic channels to behave as noisy covariates rather than as stable predictive signals.

Seen in this light, the negative dynamic result is scientifically coherent. The dynamic channels are physically motivated, but coarse; informative, but imperfect; and expensive, but not rich enough to dominate a strong static graph. The most defensible conclusion is therefore that the current dynamic encoding is not yet a better representation than the static one, not that dynamics themselves are irrelevant.

6.4 Why the direct supervised baseline was strongest

The direct supervised baseline outperformed the best variational model by a modest but consistent margin. This outcome is plausible. The supervised baseline devotes all of its capacity to predicting affinity, whereas the GraphVAE must balance reconstruction, latent regularization, and optional supervision. The variational model is therefore solving a harder optimization problem with more ways to spend capacity on objectives that are only indirectly aligned with held-out affinity prediction.

The results suggest that this trade-off is not favourable at the present data scale. On the matched 1437-complex subset, the full static baseline can exploit the graph representation directly and efficiently. The GraphVAE, by contrast, learns useful latent spaces, but its best latent representation still does not outperform the direct predictive model when judged on held-out RMSE and correlation. This does not mean the GraphVAE is without value. It still offers an interpretable latent representation, a principled reconstruction objective, and a platform for future representation-learning experiments. It does mean, however, that for the present thesis question, the simpler predictive model is the stronger empirical answer.

6.5 What the train–validation–test behaviour shows

The overfitting patterns help distinguish genuine representation weakness from a mere lack of model capacity. The unsupervised GraphVAE families exhibit small train–test gaps, but this is largely because they underfit the affinity target even on their training sets. The semi-supervised families reveal a clearer story. In particular, the semi-supervised SD families fit the training data more strongly than the corresponding S families, yet do

not improve held-out performance. This is exactly the pattern one would expect when a feature set increases model flexibility more than it increases robust signal.

The most informative comparison is between the full semi-supervised static and full semi-supervised dynamic families. The dynamic family achieves higher mean training correlation, but lower mean test correlation and a substantially larger train–test RMSE gap. The same pattern is even stronger for the interface semi-supervised dynamic family. In other words, the dynamic channels are not merely neutral; they appear to provide additional degrees of freedom that the model can exploit on the training set without converting them into better held-out predictions. This is why the overfitting story is central to interpreting the dynamic results.

6.6 Limitations and threats to validity

Several limitations should be stated explicitly. The first is dataset coverage. Although the curated PPB-Affinity subset contained 2818 complexes, the final confirmatory analysis was restricted to a matched retained subset of 1437 complexes after molecular-dynamics processing, force-feature compatibility checks, and interface-view materialization. The results are therefore not estimates on the entire curated dataset, but on the subset that the full active pipeline could process consistently.

The second limitation is physical fidelity. Missing residues are not rebuilt, heterogens are removed, all simulations use protein-only prepared systems in explicit solvent, and some experimental metadata are missing. The dynamic descriptors are therefore derived from physically meaningful but simplified systems. The third limitation is representational fidelity. The graph topology is static, the interface patch is heavily capped, the trajectories are short, and the dynamic descriptors are summary statistics rather than learned temporal representations. The fourth limitation is evaluation scope. Although the repeated outer resampling design is much stronger than a single split, it is still an internal validation on one dataset family rather than an external benchmark on an entirely different source of complexes and affinity measurements.

These limitations do not invalidate the thesis conclusions. In fact, the main conclusions are carefully phrased so that they remain valid under these limitations. What they do restrict is the scope of interpretation. The thesis provides evidence about a specific and fully described MD-to-graph pipeline. It does not provide the final word on whether dynamics can help protein–protein affinity prediction in general.

6.7 Future work

The most immediate improvement would be to revise the dynamic feature construction rather than to abandon dynamics altogether. Excluding equilibration frames from the

feature summaries would be a straightforward correction. Longer production trajectories would make the dynamic summaries less dominated by local relaxation and more sensitive to genuine conformational variability. A richer representation could also retain temporal structure instead of collapsing trajectories to means and standard deviations. Examples include time-windowed summaries, recurrent or temporal graph architectures, or graph construction that changes with the trajectory rather than remaining fixed.

On the graph side, the interface result suggests that the current patching strategy is too restrictive. A larger interface budget, a soft attention mechanism over full graphs, or a hierarchical model that combines interface-local and global context could test whether the weakness lies in the concept of interface focus or in the present hard truncation. On the modelling side, the direct supervised baseline sets a demanding but useful benchmark. Any future representation-learning model should aim either to beat this baseline directly or to offer a secondary benefit, such as transferability or interpretability, that justifies its additional complexity.

6.8 Conclusion

The thesis set out to test whether short explicit-solvent molecular dynamics could improve residue-graph prediction of protein–protein binding affinity on PPB-Affinity-derived complexes. The final answer is precise. For the implemented pipeline, the full residue graph is better than the interface patch, the current dynamic feature set does not improve robust held-out performance over the static graph, and the strongest overall model is a direct supervised baseline on the full static graph. These findings are supported by a repeated outer held-out evaluation rather than by a single favourable split.

This is a useful outcome. It demonstrates that careful benchmarking can overturn optimistic first impressions, that negative results can still be informative when they are rigorously established, and that strong static baselines remain difficult to beat. Just as importantly, it leaves a clear path forward. The thesis does not show that dynamics are irrelevant. It shows that dynamics must be represented more carefully than they were here if they are to improve affinity prediction in a reliable way.

Appendix A

Supplementary Technical Background

A.1 Molecular dynamics in plain terms

The core idea of molecular dynamics is simple: atoms are treated as particles whose positions change over time according to forces derived from a potential-energy function. If $\mathbf{r}_i$ is the position of atom i, then the force on that atom is

$$\mathbf{F}_i = -\nabla_{\mathbf{r}_i} U, \tag{A.1}$$

where U is the total potential energy of the system. In a biomolecular force field, this total energy is usually decomposed into bonded and nonbonded terms,

$$U = U_{\text{bond}} + U_{\text{angle}} + U_{\text{dihedral}} + U_{\text{elec}} + U_{\text{vdW}}. \tag{A.2}$$

The bonded terms maintain local covalent geometry, while the electrostatic and van der Waals terms model longer-range interactions between atoms that are not directly bonded.

In explicit-solvent simulation, water molecules and ions are included as ordinary particles in the simulation box. This is important because solvation contributes substantially to protein behaviour. The thermostat controls temperature, the barostat controls pressure, and periodic boundary conditions prevent the finite simulation box from behaving like an isolated vacuum droplet. Particle-mesh Ewald electrostatics are used because direct electrostatic summation over all periodic images would otherwise be too expensive for realistic systems.

For the present thesis, the goal of MD is not to compute a rigorous free energy of binding. Instead, short trajectories are used as a source of physically motivated descriptors. The intuition is that even a relatively short trajectory may reveal useful information about which residues remain hydrated, which contacts persist, and which parts of the

interface fluctuate more strongly. This is why the trajectory outputs are later reduced to residue- and edge-level summary statistics.

A.2 Residue graphs and graph neural networks in plain terms

A residue graph replaces thousands of atoms with a smaller and more interpretable set of residues. Each residue becomes a node, and edges connect residue pairs that are spatially close or interact strongly. This creates a graph that is small enough for efficient learning but still rich enough to preserve important structural context.

A graph neural network updates each node by repeatedly combining its current representation with information from neighbouring nodes. The general pattern is

$$\mathbf{h}_i^{(\ell+1)} = \phi^{(\ell)} \left(\mathbf{h}_i^{(\ell)}, \sum_{j \in \mathcal{N}(i)} \psi^{(\ell)}(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ij}) \right). \tag{A.3}$$

The message function $\psi^{(\ell)}$ says how much node j should influence node i, the update function $\phi^{(\ell)}$ combines that information with the current state of node i, and the edge features $\mathbf{e}_{ij}$ allow the strength or type of connection to matter. After several layers, the node states are pooled to form a graph-level vector that can be used for prediction.

The specific convolution used in this thesis is GINE, which extends the graph isomorphism network family to edge-feature-aware message passing [3, 4]. Intuitively, this makes it possible for the model to distinguish not just *which* residues are neighbours, but *how* they are related through distance, interaction type, or trajectory-derived contact statistics.

A.3 Variational autoencoders in plain terms

A standard autoencoder learns two maps. The encoder compresses an input into a low-dimensional latent code, and the decoder tries to reconstruct the input from that code. A variational autoencoder goes one step further: instead of producing a single latent vector, the encoder produces the parameters of a probability distribution over latent vectors, usually a Gaussian with mean μ and variance σ^2. A latent vector is then sampled as

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{A.4}$$

which is the reparameterization trick introduced by Kingma and Welling [17]. This trick allows the model to remain differentiable during training.

The VAE objective contains two parts. The reconstruction term says that the latent variable should preserve information needed to reconstruct the input. The Kullback–Leibler term says that the latent distribution should remain close to a simple prior, usually a standard normal distribution. If the KL term is too weak, the latent space can become irregular and hard to interpret. If it is too strong, the latent code can collapse and become uninformative. In practice, one often increases the KL weight gradually during training so that the model first learns to reconstruct and then learns to regularize its latent space.

In this thesis, the VAE is used as a representation-learning model on graphs. The encoder produces a graph-level latent code, and the decoder reconstructs masked node and edge features. The model is therefore not being asked to generate full protein complexes from scratch. Instead, it is being asked to learn a latent representation that is useful for reconstructing selected graph attributes and, in the semi-supervised setting, also useful for affinity prediction.

A.4 Why use ridge regression on the latent space

The final GraphVAE evaluation is performed with a ridge regressor on the exported latent means rather than with the neural affinity head alone. The reason is interpretability. If a very flexible nonlinear regressor were used after the latent space, it would be difficult to know whether good performance reflected a strong latent representation or simply a strong downstream regressor. Ridge regression is much simpler. It learns a linear mapping from latent space to affinity while penalizing large coefficients:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + +\alpha\|\mathbf{w}\|_2^2. \tag{A.5}$$

If the latent space is genuinely informative, even this simple model should perform well. If it does not, then the representation itself is probably not strong enough.

This evaluation choice also makes comparisons cleaner. The direct supervised baseline answers the question "how well can affinity be predicted directly from the graph?" The ridge probe answers the question "how predictive is the representation learned by the VAE?" These are related but not identical questions, and it is useful to keep them separate.

A.5 Why repeated outer resampling matters in practical terms

Suppose two model configurations differ by only a small amount. On one convenient train/test split, one of them may look clearly better simply by chance. If the project stopped there, the resulting scientific story could be misleading. Repeated outer resam-

pling reduces that risk. Each configuration is retrained from scratch on multiple distinct train/validation/test partitions, and conclusions are based on the aggregate pattern across those runs.

In practical terms, this means that the thesis does not rely on one best-looking experiment. Instead, it asks whether a design choice such as using the full graph, using dynamic features, or using a variational objective remains beneficial across many retrainings and many held-out subsets. This is why some results that looked promising in earlier one-split experiments no longer looked compelling in the final confirmatory matrix.

Appendix B

Archived Approaches and Project Evolution

This appendix records the main lines of work that were explored seriously during the project but did not become part of the final reported pipeline. I include them for two reasons. First, they account for a substantial part of the technical work carried out during the project, and omitting them would give an inaccurate impression of how the final direction was reached. Second, several of these branches were reasonable ideas on paper. They were not abandoned because they were obviously misguided, but because they turned out to be harder to make reliable, less competitive than hoped, or simply less suitable for a final-year thesis whose main contribution needed to be coherent, reproducible, and explainable end to end.

All of the code discussed here is preserved in the repository under `ARCHIVE`, and can also be browsed on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE`. The goal of this appendix is not to present those branches as polished alternatives to the final pipeline. The goal is to explain what they were trying to do, why they were attractive at the time, what was actually implemented, and why the project eventually moved on.

B.1 Correction learning around fast affinity baselines

One early project idea was to avoid predicting binding affinity entirely from scratch. Instead, the model could start from a fast existing estimator and learn a correction term on top of it. This is an attractive strategy because it combines a hand-designed or physics-inspired baseline with a statistical model that may learn where the baseline fails. Two branches were explored in this spirit: a contact-based baseline using PRODIGY [13] and an endpoint free-energy baseline using MM/GBSA [12].

B.1.1 PRODIGY

The PRODIGY-related code is preserved under `ARCHIVE/PRODIGY`, or on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE/PRODIGY`. This branch contains dataset parsing, chain-group normalization, estimate prefetching, and a runner that calls an external PRODIGY executable on a protein complex and parses the predicted ΔG. Conceptually, this branch was motivated by the observation that contact-based affinity models are simple, interpretable, and surprisingly competitive for some protein–protein benchmarks. That motivation is well grounded in the literature: Vangone and Bonvin showed that interfacial contact counts and related structural descriptors can correlate meaningfully with experimental binding affinity in protein–protein complexes [13].

In practical terms, the branch was intended to answer a simple question: if a contact-based structural predictor already gives a rough estimate, can a second model learn the residual error more easily than predicting affinity from raw structural input alone? This is a sensible idea, especially for an undergraduate project, because it keeps one foot in an interpretable baseline while still allowing machine learning to add value.

However, the preserved outputs in the repository were not convincing enough to justify making PRODIGY the core of the thesis. The committed exploratory estimates under `src/bridging/generatedData/PRODIGY` are sparse and were not produced under the final confirmatory protocol, so they should be treated as indicative rather than definitive. Even so, they point in the same direction as the qualitative decision that was made at the time. On the small preserved `PPB_Affinity_diverse100` exploratory subset, the matched cases in `PPB_Affinity_diverse100_prodigy_estimates.csv` give a Pearson correlation of approximately $r = 0.31$ and RMSE of approximately 3.17 kcal/mol against the experimental ΔG values. On the preserved antibody–antigen subset, only four matched cases remain, and the corresponding correlation is again weak. In addition, the log file `src/bridging/generatedData/PRODIGY/prodigy_failures.log` contains many execution failures, which made this branch operationally awkward as well as scientifically unconvincing.

For those reasons, PRODIGY was kept as an archived baseline branch rather than developed into the main thesis contribution. The underlying idea still made sense, but it did not provide a strong enough foundation for the final story. In particular, a correction-learning thesis built on a weak baseline would have been much harder to motivate than a direct comparison between static and MD-derived graph representations.

B.1.2 MM/GBSA

The MM/GBSA branch is preserved under `ARCHIVE/MMGBSA`, or on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE/MMGBSA`. This branch includes AmberTools-based input preparation, sharded HPC launchers, prefetched estimate

handling, and a runner that can execute MM/GBSA either from a single structure or from a supplied trajectory. The motivation here was again a correction-learning or comparison-baseline idea, but with a stronger physical flavour. MM/GBSA occupies an interesting middle ground in the literature: it is more physics-based than purely empirical scoring, but much cheaper than rigorous alchemical free-energy calculations [12]. That makes it a natural candidate baseline when one wants to ask whether machine learning can improve on an established approximate method.

The attraction of this path was obvious. If an endpoint free-energy approximation already captures some part of the binding signal, then learning a residual around that estimate could be more data efficient than learning affinity directly from raw structural descriptors. It also would have provided a clear bridge between classical computational chemistry and machine learning, which would have been thematically appealing for this project.

In practice, though, this branch never matured into a clean final result set comparable to the GraphVAE matrix reported in the main body of the thesis. The implementation is real and substantial, but the repo does not contain a final preserved metrics snapshot for MM/GBSA in the same directly auditable form as the final GraphVAE results. That matters. For the final thesis, I wanted the central empirical claims to be tied to artifacts that were still present, inspectable, and rerunnable from the repository state. MM/GBSA did not meet that bar by the time the project direction settled.

There is also a scientific reason why this branch became less attractive over time. As reviewed by Genheden and Ryde, MM/PBSA and MM/GBSA are useful but heavily approximation-dependent methods whose performance can vary strongly with the system, the solvent model, the entropy treatment, and implementation details [12]. That does not make them bad methods, but it does mean they are a difficult backbone for a thesis that also aims to compare machine-learning representations in a controlled way. In the end, MM/GBSA remained valuable as an idea and as preserved engineering work, but it was not the right centre of gravity for the final thesis.

B.2 The Cα contact-map branch

The oldest major modelling branch in the repository is the contact-map pipeline preserved under `ARCHIVE/CA_contact_map`, or on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE/CA_contact_map`. This branch converts selected interface residues into regular two-dimensional tensors built from Cα–Cα contact and distance channels, then trains a convolutional variational autoencoder and downstream regression model on those tensors.

The motivation for this branch was straightforward. A contact map gives a fixed-size, image-like object that is much easier to process with standard convolutional architectures

than a variable-size protein graph. That design choice is common in adjacent areas of structural machine learning, especially in protein–ligand binding work where voxelized or tensorized spatial representations have been used successfully, for example in Pafnucy [15]. For a project that was still trying to find a stable representation, this seemed like a sensible starting point: it imposed a strong geometric regularity on the data and made the learning problem easier to package.

The preserved code shows exactly what this branch was doing. Interface residues were selected from two chain groups using either frame-0 geometry or a more stable early-trajectory criterion in `ARCHIVE/CA_contact_map/src/bridging/featurization/interface.py`. The resulting residue subsets were converted into soft contact and clipped distance channels in `ARCHIVE/CA_contact_map/src/bridging/featurization/contact_map.py`. A set-based convolutional variational autoencoder was then trained on these tensors in `ARCHIVE/CA_contact_map/src/bridging/ml/model.py`, followed by downstream regression in `ARCHIVE/CA_contact_map/src/bridging/regression`.

This was not a nonsense design. In fact, it solved several engineering problems cleanly. Fixed tensor shape makes batching easy. The representation is compact. The interface selection logic is explicit. The model class is relatively small and interpretable. The problem is that this convenience comes at a scientific cost. By projecting a heterogeneous protein–protein interface onto a fixed Cα contact tensor, the branch discarded a large amount of residue identity, side-chain, and graph-topological information. It also made the exact representation depend heavily on patch-selection and ordering choices. Once the graph pipeline was in place, the contact-map branch started to look like an over-compressed view of the same problem rather than the right abstraction for it.

So this branch was abandoned not because it was broken, but because it became hard to justify as the central representation once a residue-graph alternative existed. The graph formulation retained more chemically meaningful structure, handled variable-size complexes more naturally, and ended up being easier to connect to the MD-derived features that became the core of the final project.

B.3 The PPB-inspired augmentation and spatiotemporal branch

The PPB-inspired deep-learning branch is preserved under `ARCHIVE/PPB_Affinity_augmentation`, or on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE/PPB_Affinity_augmentation`. This is probably the largest archived branch after the final pipeline itself. It contains three related modes: frame-level training, trajectory-level training, and an ensemble that combines predictions from several learned views. It also vendors a substantial external model stack under `ARCHIVE/PPB_Affinity_augmentation/`

`src/BaselineModel`.

The motivation here was different from the PRODIGY and MM/GBSA branches. This branch was not mainly about correcting a baseline. It was about taking the idea of MD-informed learning more directly and more aggressively. The PPB-Affinity dataset paper explicitly argues for AI-based protein drug discovery on protein–protein complexes [1], and more recent protein–ligand work has shown that spatio-temporal learning directly on MD-derived examples can be effective in at least some settings [16]. That made it tempting to try a more direct learning pipeline over frames and trajectories rather than reducing the MD to a small set of handcrafted graph features.

The archived implementation shows that this branch went well beyond a sketch. The frame mode in `ARCHIVE/PPB_Affinity_augmentation/src/bridging/ppb/frame_mode.py` trains a supervised structural model on sampled frame-level examples, with patch selection and augmentation. The trajectory mode in `ARCHIVE/PPB_Affinity_augmentation/src/bridging/ppb/trajectory_mode.py` uses a temporal model built on top of frame embeddings, including a bidirectional GRU over frame sequences. The ensemble mode in `ARCHIVE/PPB_Affinity_augmentation/src/bridging/ppb/ensemble_mode.py` fits a linear combiner across baseline, frame, and trajectory predictions. In other words, this branch represented a serious attempt at frame-aware and time-aware learning rather than a minimal proof of concept.

Why was it not carried through as the final thesis? The honest answer is that this was mainly a project-management pivot rather than a clean negative result. The branch was ambitious and interesting, but it also introduced several layers of complexity at once: a large vendored model stack, frame sampling decisions, temporal modelling decisions, augmentation policy, and evaluation logic distinct from the rest of the project. For a final-year thesis, that was a lot to own and explain rigorously. The later MD-to-graph pipeline was narrower, easier to audit, and easier to connect to concrete scientific questions such as "does the full graph help more than an interface patch?" and "do these particular dynamic features improve held-out performance?" Those questions fit the eventual scope of the thesis better.

So this branch should not be read as "tried and failed." A more accurate reading is "worth trying, partially implemented, but eventually superseded by a more focused and more explainable project direction." I think that distinction matters.

B.4 Exploratory dataset curation utilities

The final archived workstream is `ARCHIVE/Data_curation`, or on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE/Data_curation`. These scripts cover earlier filtering decisions, partner-size prefetching, subset selection, diverse-sample construction, and older cleaning steps for intermediate datasets.

This material is less glamorous than the modelling branches, but it was still important. Before the final broad, pair-unique PPB-Affinity-derived dataset was fixed, there was a real question of which subset definitions were sensible and computationally feasible. Some branches focused on antibody–antigen cases, some on more diverse subsets, and some on excluding complexes whose partners were too large for a given representation or runtime budget. Those decisions directly affected what was possible to run, especially in the earlier stages of the project.

These scripts are archived simply because the final pipeline now has a stable dataset definition and no longer depends on those exploratory filters. They still matter for provenance, though, because they show that the final dataset was not chosen in a vacuum. It emerged from earlier attempts to balance biological scope, computational tractability, and representation design.

B.5 What the archived branches contributed

Although these branches did not survive into the final reported pipeline, they influenced the final thesis in concrete ways. The correction-learning branches clarified that fast structure-only or endpoint baselines were worth comparing against, but were not enough to carry the project on their own. The contact-map branch showed that enforcing a regular tensor shape made the problem easier to package but also removed too much useful structure. The PPB-inspired augmentation branch showed that direct frame- and trajectory-level learning was possible, but that it would be difficult to make the resulting thesis as auditable and focused as it needed to be. The data-curation utilities helped narrow the problem to a dataset definition that was broad enough to be interesting and structured enough to support repeated evaluation.

In that sense, the archived material is not just dead code. It is the record of how the final thesis question became sharper. The thesis eventually settled on a simpler but more defensible comparison: static versus MD-derived residue-graph features on a matched protein–protein affinity dataset, evaluated under repeated held-out resampling. That final formulation was reached by ruling out, narrowing, or deferring several other plausible directions, and the `ARCHIVE` directory is where that history now lives.

Appendix C

Repository and Implementation Availability

The implementation for this thesis is available in the public repository `https://github.com/RohanDevereux/Bridging`. Rather than reproducing large blocks of source code in the thesis itself, this chapter points to the parts of the repository that correspond directly to the pipeline described in the main text.

The active code path used for the final results is concentrated in a small part of the repository. Molecular-dynamics preparation and simulation live in `src/bridging/MD` and can also be browsed directly on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/MD`. Graph preparation, including DeepRank2 integration, dynamic feature extraction, force-proxy construction, and graph-view materialization, lives in `src/bridging/graphvae/prep` and `src/bridging/graphvae/runners`, corresponding to `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/prep` and `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/runners`. Model definitions, dataset preprocessing, the supervised baseline, and the latent-space regression code live in `src/bridging/graphvae/ml`, corresponding to `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/ml`. Shared parsing and affinity-conversion utilities live in `src/bridging/utils`.

The key committed artifacts used in the final thesis are also present in the repository snapshot. The curated dataset used by the active pipeline is `src/bridging/processedData/PPB_Affinity_broad_pairuniq_train80_test20.csv`. The matched full and interface graph-record files used in the final confirmatory resampling study are `graphvae_force_v3_results/src/bridging/generatedData/graphvae/PPB_Affinity_broad_pairuniq_train80_test20_views_force_mdiface_v3/md_closest_pair_patch/graph_records_full.pt` and `graphvae_force_v3_results/src/bridging/generatedData/graphvae/PPB_Affinity_broad_pairuniq_train80_test20_views_force_mdiface_v3/md_closest_pair_patch/graph_records_interface.pt`. The compact summary of the

final 52-configuration resample matrix is `RESULTS/final_resample_matrix_summary.csv`. Chapters 5 and 6 are written against these exact artifacts.

The repository also contains HPC launch scripts under `hpc`. These scripts are useful for rerunning the larger stages of the pipeline, but they are wrappers around the Python entry points rather than a separate source of logic. For understanding or auditing the implementation, the Python modules listed above are the important locations.

Finally, the repository contains an `ARCHIVE` directory, also visible at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE`. This material is intentionally preserved because it records substantial work that shaped the final direction of the project, even when that work did not end up in the final reported pipeline. Appendix B explains those archived branches, their motivations, and why they were ultimately not part of the final thesis contribution.

Bibliography

[1] H. Liu, P. Chen, X. Zhai, K.-G. Huo, S. Zhou, L. Han, and G. Fan, Scientific Data **11**, 1316 (2024).

[2] G. Crocioni, D. L. Bodor, C. Baakman, F. M. Parizi, D.-T. Rademaker, G. Ramakrishnan, S. A. van der Burg, D. F. Marzella, J. M. C. Teixeira, and L. C. Xue, Journal of Open Source Software **9**, 5983 (2024).

[3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, in *International Conference on Learning Representations* (2019), URL `https://openreview.net/forum?id=ryGs6iA5Km`.

[4] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, International Conference on Learning Representations (2020), URL `https://openreview.net/forum?id=HJlWWJSFDH`.

[5] M. Fey and J. E. Lenssen, ICLR Workshop on Representation Learning on Graphs and Manifolds (2019), URL `https://arxiv.org/abs/1903.02428`.

[6] K. Henzler-Wildman and D. Kern, Nature **450**, 964 (2007).

[7] P. Eastman, R. Galvelis, R. P. Peláez, C. R. A. Abreu, S. E. Farr, E. Gallicchio, et al., The Journal of Physical Chemistry B **128** (2024).

[8] R. T. McGibbon, K. A. Beauchamp, M. P. Harrigan, C. Klein, J. M. Swails, C. X. Hernández, C. R. Schwantes, L.-P. Wang, T. J. Lane, and V. S. Pande, Biophysical Journal **109**, 1528 (2015).

[9] J. Jankauskaitė, B. Jiménez-García, J. Dapkūnas, J. Fernández-Recio, and I. H. Moal, Bioinformatics **35**, 462 (2019).

[10] G. M. Torrie and J. P. Valleau, Journal of Computational Physics **23**, 187 (1977).

[11] Z. Cournia, B. Allen, and W. Sherman, Journal of Chemical Information and Modeling **57**, 2911 (2017).

[12] S. Genheden and U. Ryde, Expert Opinion on Drug Discovery **10**, 449 (2015).

[13] A. Vangone and A. M. J. J. Bonvin, eLife **4**, e07454 (2015).

[14] R. Wang, X. Fang, Y. Lu, and S. Wang, Journal of Medicinal Chemistry **47**, 2977 (2004).

[15] M. M. Stepniewska-Dziubinska, P. Zielenkiewicz, and P. Siedlecki, Bioinformatics **34**, 3666 (2018).

[16] P.-Y. Libouban, C. Parisel, M. Song, S. Aci-Sèche, J. C. Gómez-Tamayo, G. Tresadern, and P. Bonnet, Bioinformatics **41**, btaf429 (2025).

[17] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, arXiv preprint arXiv:1312.6114 (2014), URL `https://arxiv.org/abs/1312.6114`.

[18] R. Gómez-Bombarelli, J. N. Wei, D. Duvenaud, J. M. Hernández-Lobato, B. Sánchez-Lengeling, D. Sheberla, J. Aguilera-Iparraguirre, T. D. Hirzel, R. P. Adams, and A. Aspuru-Guzik, ACS Central Science **4**, 268 (2018).

[19] T. N. Kipf and M. Welling, arXiv preprint arXiv:1611.07308 (2016), URL `https://arxiv.org/abs/1611.07308`.

[20] M. Simonovsky and N. Komodakis, International Conference on Artificial Neural Networks pp. 412–422 (2018).

[21] OpenMM Developers, *Pdbfixer*, `https://github.com/openmm/pdbfixer` (2026), accessed 2026-03-17.

[22] J. A. Maier, C. Martinez, K. Kasavajhala, L. Wickstrom, K. E. Hauser, and C. Simmerling, Journal of Chemical Theory and Computation **11**, 3696 (2015).

[23] L.-P. Wang and T. J. Martinez, The Journal of Physical Chemistry Letters **5**, 1885 (2014).

[24] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Journal of Machine Learning Research **12**, 2825 (2011), URL `https://jmlr.org/papers/v12/pedregosa11a.html`.